TripSync

Wspólne wyjazdy. Plan, zadania i rozliczenia - także offline.

Temat i cel projektu

Aplikacja Android do organizacji wyjazdów grupowych z pracą offline, synchronizacją danych i rozliczaniem wspólnych wydatków. Projekt dla trzech osób rozwija wymagania podstawowe o współpracę wielu użytkowników, kontrolę dostępu, wykrywanie konfliktów edycji i algorytm rozliczeń.

Użytkownik tworzy wyjazd, zaprasza grupę kodem lub linkiem, a następnie wspólnie z uczestnikami układa plan, przydziela zadania i zapisuje koszty. Dane są dostępne po ponownym uruchomieniu aplikacji i bez połączenia z internetem.

Zakres funkcjonalny

Wyjazdy i uczestnicy. Konta użytkowników, tworzenie, odczyt, edycja i usuwanie wyjazdów; zaproszenia; role organizatora i uczestnika.

Harmonogram. Punkty planu z datą, godziną, miejscem i opisem; zmiana kolejności, edycja, usuwanie i oznaczanie realizacji.

Propozycje i głosowanie. Dodawanie propozycji atrakcji, głosowanie oraz przenoszenie wybranych propozycji do harmonogramu.

Zadania i pakowanie. Wspólne listy, przypisywanie odpowiedzialnych osób oraz oznaczanie wykonania.

Wydatki i rozliczenia. Kwota, płatnik i uczestnicy kosztu; podział równy lub według kwot; salda i propozycje przelewów ograniczające liczbę rozliczeń.

Tryb offline i synchronizacja. Lokalny zapis i kolejka operacji; synchronizacja po odzyskaniu sieci, widoczny status oraz wybór wersji przy konflikcie edycji.

Przypomnienia. Lokalne powiadomienia o zbliżających się punktach harmonogramu i zadaniach.

Interfejs i granice zakresu

Osiem głównych ekranów: logowanie/rejestracja, lista wyjazdów, podsumowanie, harmonogram, szczegóły/edycja punktu, propozycje i głosowanie, zadania/pakowanie oraz wydatki/rozliczenia. Nawigacja zachowuje właściwy stan ekranów.

Jedna waluta na wyjazd. Zakres nie obejmuje czatu, płatności, rezerwacji ani pełnych map offline. Algorytm proponuje uproszczone rozliczenia bez gwarancji globalnie minimalnej liczby przelewów.

TripSync

Wspólne wyjazdy. Plan, zadania i rozliczenia - także offline.

Technologia i architektura

Kotlin, Android Studio, stabilne Android SDK. Interfejs: Jetpack Compose i Material Design 3. Architektura: MVVM z warstwą domenową, repozytoriami i rozdzieleniem odpowiedzialności.

Dane i sieć: Room (SQLite) jako lokalne źródło danych dla ekranów; backend z bazą danych i API REST; Retrofit do komunikacji. Coroutines i Flow obsługują operacje asynchroniczne i stan, WorkManager synchronizację w tle, DataStore ustawienia, a Navigation Compose przejścia między ekranami.

Synchronizacja: operacje mają identyfikatory zabezpieczające przed powieleniem po ponownym wysłaniu. Wersjonowanie rekordów umożliwia wykrycie równoczesnych zmian. Usunięcia są przekazywane między urządzeniami; cofnięcie uprawnień blokuje dalszy dostęp na backendzie.

Jakość, bezpieczeństwo i dostępność

Walidacja formularzy, kwot, dat i podziałów kosztów; obsługa błędów sieci, bazy i sesji z czytelnymi komunikatami oraz możliwością ponowienia operacji. Obliczenia pieniężne zachowują dokładność do najmniejszej jednostki waluty.

Kontrola dostępu do danych wyjazdu po stronie backendu; komunikacja HTTPS i bezpieczne przechowywanie danych uwierzytelniających z wykorzystaniem Android Keystore. Dane kont są odseparowane, a wylogowanie usuwa lokalne dane sesji.

Responsywny interfejs dla różnych rozmiarów ekranów; odporność na zmianę konfiguracji; operacje bazy i sieci poza głównym wątkiem. Dostępność: opisy elementów, właściwy kontrast i obsługa większego rozmiaru tekstu. Kod modularny, umożliwiający dalszą rozbudowę.

Podział pracy i realizacja

Osoba 1: konta, backend, wyjazdy, zaproszenia i kontrola dostępu.
Osoba 2: harmonogram, głosowanie, zadania, nawigacja i przypomnienia.
Osoba 3: wydatki, algorytm rozliczeń, lokalna baza i synchronizacja.
Model danych i kontrakt API są ustalane wspólnie; każda osoba odpowiada za testy i dokumentację swojego modułu.

Etapy: (1) model danych i aplikacja lokalna, (2) konta i współdzielenie, (3) synchronizacja i konflikty, (4) głosowanie, przypomnienia, testy i dokumentacja.

Weryfikacja i materiały końcowe

Testy jednostkowe: podział kosztów, salda, walidacja i konflikty. Testy integracyjne: utrata sieci, ponowienie operacji, synchronizacja usunięć oraz uprawnienia. Wybrane testy UI: dodanie wydatku, nawigacja i odtworzenie danych po ponownym uruchomieniu.

Oddanie projektu: repozytorium Git z historią zmian, instrukcja uruchomienia aplikacji i backendu, opis architektury i modelu danych, dokumentacja funkcjonalna oraz wyniki testów. Demonstracja na dwóch urządzeniach obejmuje pracę offline, synchronizację, konflikt edycji i rozliczenie grupy.